# Midterm: Identifying LLMs from Their Responses

Shreya Sreehesh
CMPSC 448
Midterm Project
10/11/2026

## Problem Definition

As large language models (LLMs) become increasingly common, it is becoming more difficult to distinguish text generated by different AI systems. Models such as ChatGPT, Claude, Qwen, Llama, Gemini, and DeepSeek produce responses to the same questions, but their outputs can contain different writing styles, formats, and other stylistic choices. This project was meant to develop and evaluate deep learning models that predict which LLM generated a response. A convolutional neural network and recurrent neural network models are implemented to classify which model generates a certain response. The first question the project explored was how accurately can the models identify the LLM that generated a response using only the response text? The second question the project explored was can the models identify the LLM using only the prompt, only the response, and both prompt and response given to it?

## Dataset

The dataset used was TXD-22, “A Large-Scale Benchmark Dataset” by mdsadiqiqbal on Kaggle. It has 66,000 rows with three columns which are Question, Answer, and Source. Question is the prompt, Answer is the generated response, and Source is the large language model used. This dataset includes 22 different types of models. I have selected 6 sources to focus on for this project. These sources are ChatGPT, Claude, Gemini, Llama, Qwen, and DeepSeek. The data is split 70% train, 15% validation, 15% test. All answers to the same prompt stay in one split, which prevents the same prompt from appearing in different splits. Any duplicated answers were removed, and only questions that all six chatbots answered were kept, so each chatbot has the same number of examples. This left 17,724 rows, split into 12,402 training, 2,659 validation and 2,663 test rows. The CSV had UTF-8 and cp1252 bytes, so it decodes as UTF-8 and falls back to cp1252 for undecodable bytes. One limitation of the dataset is that every prompt is repeated once for each of the six source models, so the prompt carries no information about which model wrote the response.

https://www.kaggle.com/datasets/mdsadiqiqbal/txd-22-a-large-scale-benchmark-dataset/data

## CNN and RNN implementation

The text was tokenized into words while preserving capitalization, punctuation, and line breaks. Tokens were split with a regular expression, and line breaks were kept as a special <NL> token. The vocabulary also has <pad>, <unk> (for words not in the vocabulary) and <sep> tokens. For the prompt-plus-response setting, the prompt and the response were joined with <sep> so the model can tell where the prompt ends. Sequences longer than the maximum length were cut. A vocabulary of 40,000 tokens was created using the training data, with each token converted into a numerical index. Sequences were padded to a consistent length for model input. The maximum prompt length was set to 64 tokens and the maximum response length to 400 tokens.

The CNN used convolutional layers to identify patterns in the text. The model converted the tokens 128-dimensional embeddings, then it applied three parallel one-dimensional convolutional layers with filter sizes of 3, 4, and 5 using 100 filters for each size. The embedding layer ignores the padding token, and the embedding weights were limited to a maximum norm of 5. The CNN has 5,275,706 trainable parameters. These filters helped identify patterns across groups of neighboring tokens. ReLU activation was applied, and max-over-time pooling selected the strongest feature detected by each filter. The resulting features were concatenated, passed through a dropout layer with a rate of 0.5 to help reduce overfitting, and fed into a final linear layer that predicted one of the six LLM classes. The CNN was trained using the Adadelta optimizer with a learning rate of 0.25 and a rho value of 0.95. Cross-entropy loss was used to measure classification error.

RNN was implemented using a bidirectional Long Short-Term Memory (LSTM) network. LSTM processes sequences to learn relationships between tokens across the text. The model used 128 hidden units. Since it is bidirectional, it processes the text from both the beginning and the end. The LSTM was trained using the Adam optimizer with a learning rate of 0.001 and negative log-likelihood loss. Gradient clipping with a maximum norm of 3 was applied to help prevent excessively large gradients during training. The same 128-dimensional embedding was used, with a dropout of 0.4 on the embeddings. The final forward and backward hidden states were concatenated, passed through dropout and a linear layer to predict one of the six classes. The LSTM has 5,385,734 trainable parameters. Packed sequences were used to account for the different lengths of text sequences.

CNN and LSTM were trained for a maximum of 10 epochs using a batch size of 50. The random seed was set to 42 to support reproducibility. After each epoch, the models

were evaluated on the validation set using validation loss and accuracy. The patience of three epochs was implemented for early stopping. Training stopped when validation accuracy failed to improve for three consecutive epochs. Accuracy and macro F1 score were calculated to measure the classification performance across the six classes. Macro F1 was used because it averages the F1 score for each LLM, so a class the model never predicts lowers the scores. Training and validation loss curves were saved to examine learning behavior. To compare input settings, the models were trained with prompt only inputs, response only inputs, and prompt and response inputs. This resulted in six experiments and allowed CNN and LSTM to be compared under the same input conditions.

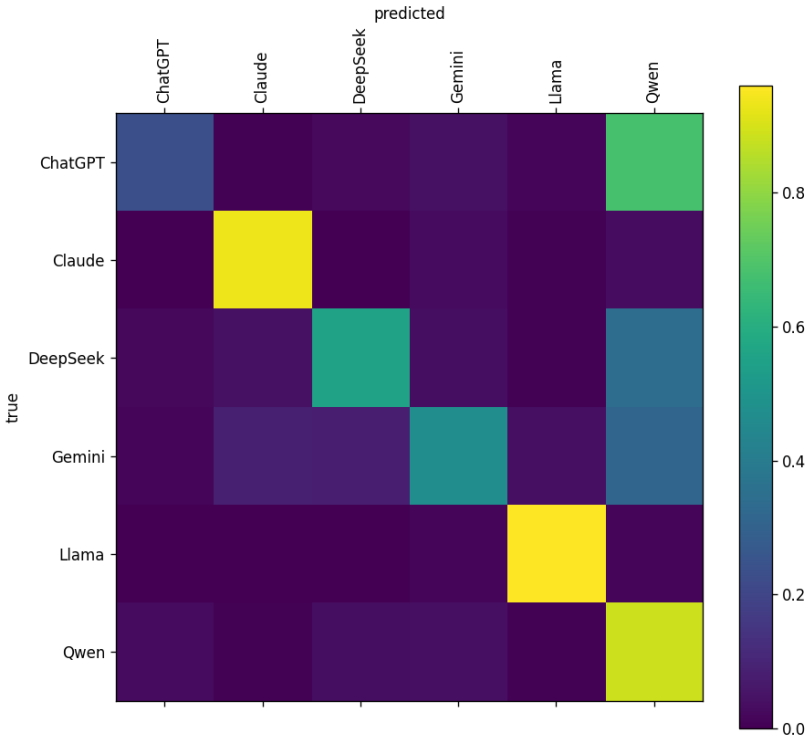
## Results and Findings

CNN and RNN (LSTM) were able to identify which LLM generated a response based on the output. LSTM performed better than CNN. Using the responses as input, the CNN has a 64.33% accuracy and a macro F1 score of 0.6088. LSTM has a 79.76% accuracy and a macro F1 score of 0.7964 across all six LLM classes. These results indicate that the output response has helped distinguish between different types of LLMs. LSTM outperformed CNN by 15.4 points in accuracy, meaning that LSTM was more successful in accurately classifying the text. The confusion matrices also show that LSTM was more successful at correctly classifying responses across the LLMs. Overall, the results suggest that the LSTM was more effective than the CNN at identifying patterns in the generated text for this dataset. However, the models still misclassified by about 20% for LSTM and 36% for CNN of the answers.

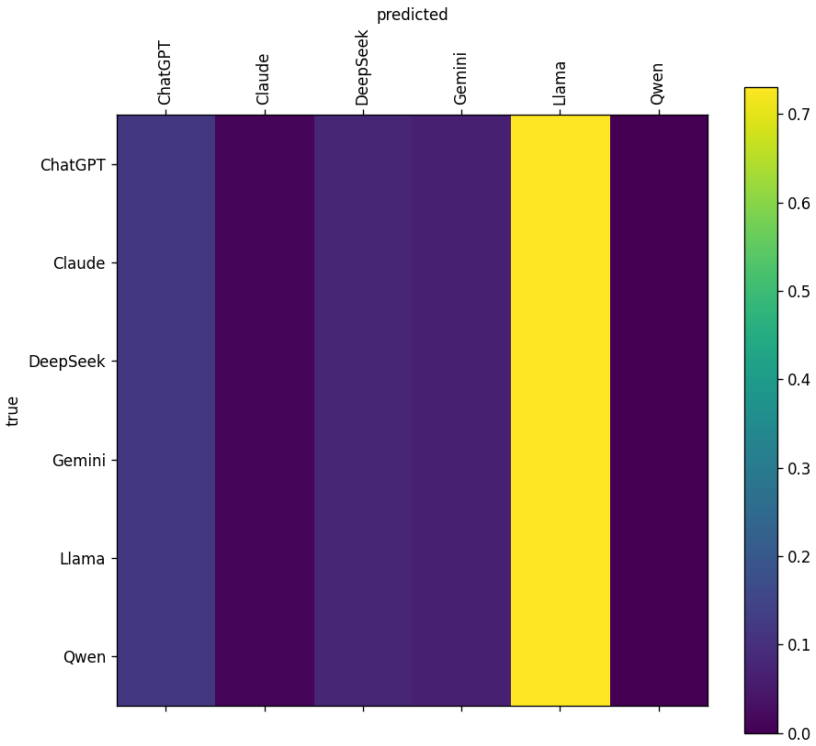
From the results and findings of this project, it seems difficult to predict which LLM generated a response from the user's prompt. CNN has a 16.79% accuracy and macro F1 score of 0.1048. The LSTM had a 16.64% accuracy and macro F1 score of 0.1042. Additionally, the confusion matrices show that both models predicted a few LLM classes rather than all six. LSTM predicted Claude for most prompts and did not even predict Llama or Qwen. These results suggest that the input prompt doesn't provide enough information for the models to identify which LLM generated a response. Including the input prompt with the output response did not improve the classification. For CNN, accuracy changed from 64.33% with the response only to 67.37% with both the prompt and the response. The 3.04 point difference is within the noise of the run. For LSTM, the accuracy decreased from 79.76% to 76.72% when the prompt was included, which suggests that the prompt added no useful information for the classification. Prompt-only classification did not work surprisingly well, but if it did, it could indicate that different LLMs received different types of prompts. In this dataset, however, every LLM answered the same questions, so the prompt

carries no information about which LLM wrote the response, and the models cannot do better than chance.

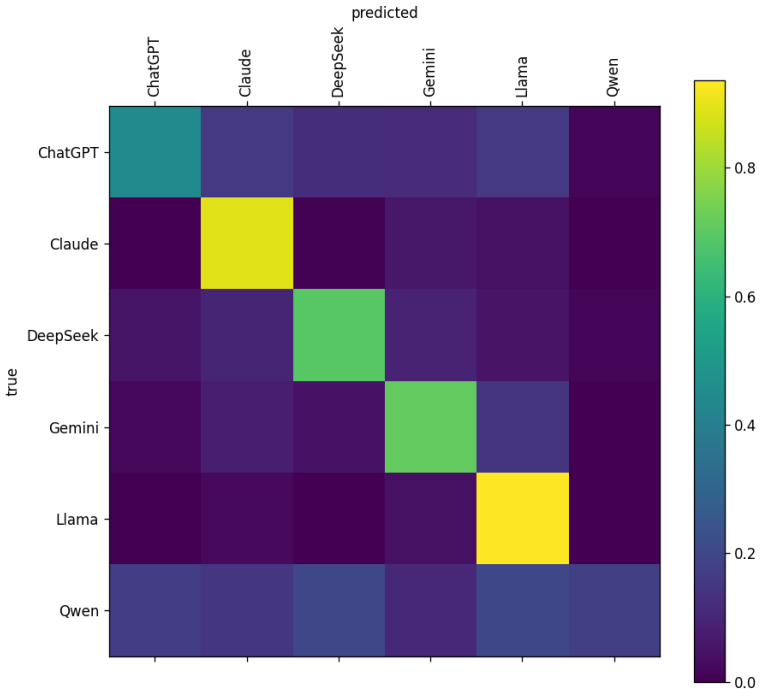
| Input | Model | Test Accuracy | Macro F-1 |
|---|---|---|---|
| Response | CNN | 64.33% | 0.6088 |
| Response | LSTM | 79.76% | 0.7964 |
| Prompt | CNN | 16.79% | 0.1048 |
| Prompt | LSTM | 16.64% | 0.1042 |
| Response and Prompt | CNN | 67.37% | 0.6635 |
| Response and Prompt | LSTM | 76.72% | 0.7641 |



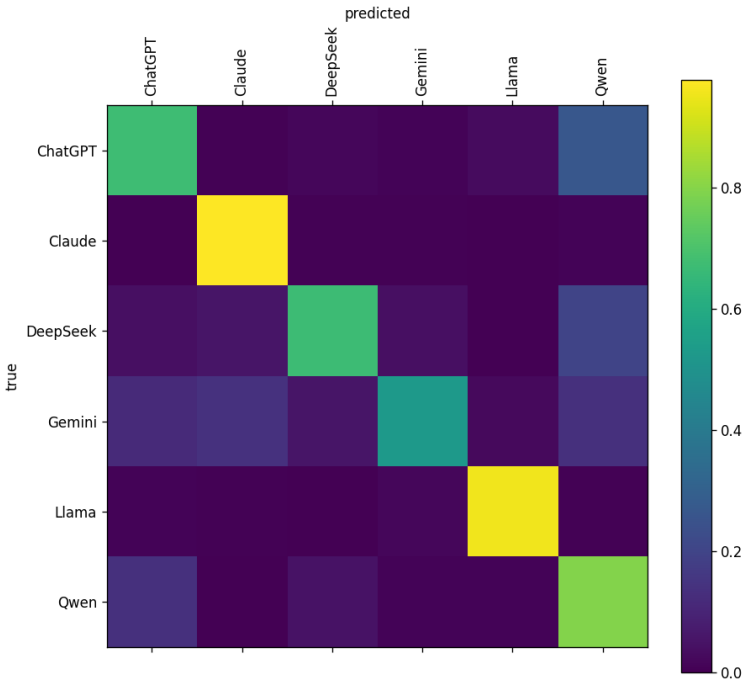

CNN with Both Prompt and Response



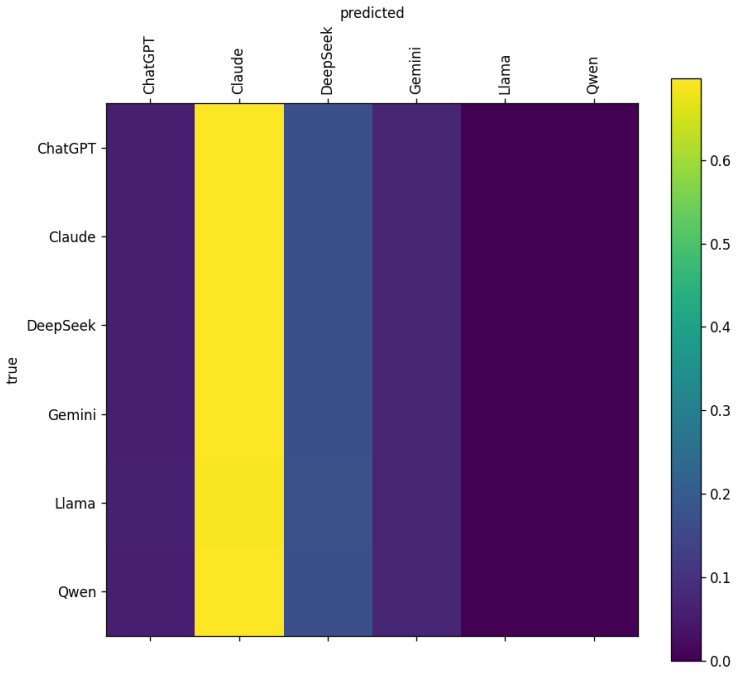

CNN with Only Prompt



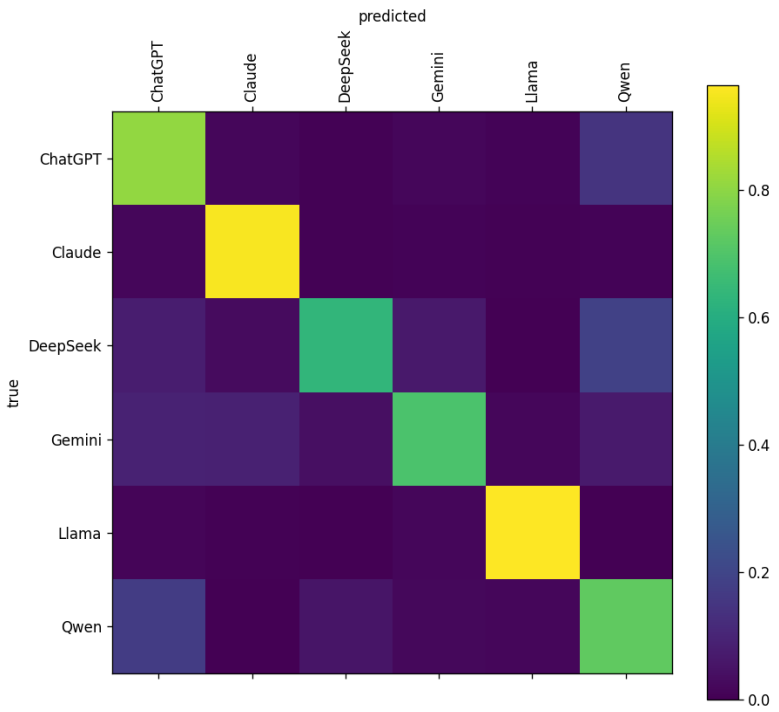

CNN with Only Generated Response




LSTM with Both Prompt and Response




LSTM with Only Prompt




LSTM with Only Generated Response

## Further Analysis

It seems based on the model's performance, that prompt-only classification is based on chance. In TXD-22 every chatbot answered the same question, and the split keeps all six answers to a question together. In the training set, each prompt therefore appears six times, once with each label. The label is statistically independent of the prompt, so the best possible classifier outputs the same probability for every class and scores 1/6.

The collapsed predictions follow from the same fact. One or two classes win everywhere, Llama for CNN and Claude for LSTM. Macro-F1 falls to about 0.10 because a never predicted class has F1 = 0. The prompt carries no label information, so it can only add noise. The models also performed better on some LLMs than on others. LSTM finds Llama (F1 = 0.96) and Claude (F1 = 0.91) almost every time and has the most trouble with Qwen (F1 = 0.68), DeepSeek (F1 = 0.73) and ChatGPT (F1 = 0.74). Its three largest confusions are DeepSeek labeled Qwen, Qwen labeled ChatGPT, and ChatGPT labeled Qwen. A possible reason is that these three LLMs write in a more similar style, but this was not tested.

## Lessons and Experience

Although both the CNN and LSTM were trained to identify which of the six LLMs generated a response, they processed the input text differently. CNN used three parallel convolutional layers with different kernel sizes to identify patterns. Max pooling selected the strongest features from the text which were then passed through fully connected layer for classification. LSTM used a bidirectional recurrent architecture to process text sequences in both forward and backward directions. This allowed it to use information from the sequence when making predictions. The CNN was trained using Adadelta while the LSTM used Adam with gradient clipping. When comparing the results, CNN had received a lower accuracy of 64.33% compared to LSTM 79.76% accuracy. This suggests that LSTM was more effective in classifying the generated text between the six LLMs. One possible explanation is that LSTM's ability to process word sequences in both directions helped it capture useful patterns in the responses. The results cannot establish the exact reason or the performance difference. I also learned that including more information does not always improve a model's performance. One can assume that including the prompt and the response will always improve the ability to classify the generated text, but this proved to be wrong for LSTM as the accuracy decreased with the inclusion of the prompt when running the model. The response text contained enough information to identify a pattern. It also reconfirmed that you cannot classify the LLM model that generated text based on the prompt input. This makes sense because every LLM was given the same

prompts. There could be patterns in the format or type of question asked to each LLM, but in this dataset the prompts are identical across LLMs, so the prompt cannot be used to classify the LLM. I also learned that early stopping with the best checkpoint restored, and testing only once, keeps the results honest. The LSTM gave identical results in every run, but the CNN varied, so one run is not enough evidence for a small difference.